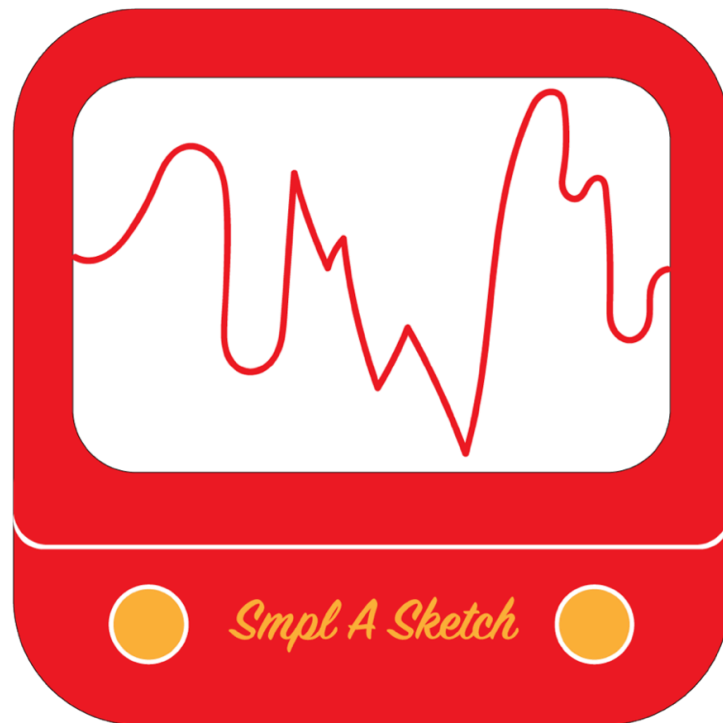

Electronic Product Design Final Report

Braden Cantor-Goldner



12/10/2025

What Is It?

The SmplASketch is a draw your own wavetable synth device taking inspiration from the much larger Oramics machines. The physical device has a slot for plexiglass slides to be inserted containing user drawn soundwaves and LFOs and the companion desktop app is able to use the data from the slides to create exportable samples. The physical device also allows you to customize pitch, envelope length, and in software there is much more additional customization including a mode that turns images into groupings of sound waves.

Video Tutorials: [Youtube Playlist](#)

Problems Encountered/Future Plans

Hardware Problems:

The biggest problems I had from this project were relatively early on. My original idea for the project was to have this controller be wireless and to use a Raspberry Pi Zero W running a processing program with no UI just for control. There came a point in which it was clear that the Raspberry Pi was going to be too weak to do all of the image and audio processing I wanted to do simultaneously. In addition, conceptually there was a gap of how I was going to get the samples off of the device. I ended up just writing a desktop app from scratch using Processing on my Mac and using a Teensy as a fancy keyboard input essentially.

The original reason I didn't want to use the Teensy is because it has no camera capabilities. I was eventually able to get around that by gutting a cheap webcam and plugging that and the Teensy into a usb hub inside of the enclosure. I then did all of the processing on the computer, not at all on the Teensy.

The only major hardware problem later in the project was that I accidentally made a mistake when expanding the key cuts in my illustrator file and it caused my enclosure to not fit with standard keycaps so I ended up having to custom design and 3D print keycaps. That was okay ultimately since I was considering custom making my keycaps anyways.

Software Problems:

All code problems were mostly minor bugs with one exception. When I was trying to add in the LFO editor, the built in Processing library didn't have anything for LFO's. I tried to create my own library by modulating the relative sample rate of the original wave but it just killed the program because it was trying to change audio variables faster than the functions could run. I had to rewrite my entire audio code base using the Minim library for Java/Processing to eventually get LFO control to work. I should have just used Minim from the start as it gave me way more control and also allowed me to finally upgrade from an ASR envelope to an ADSR envelope.

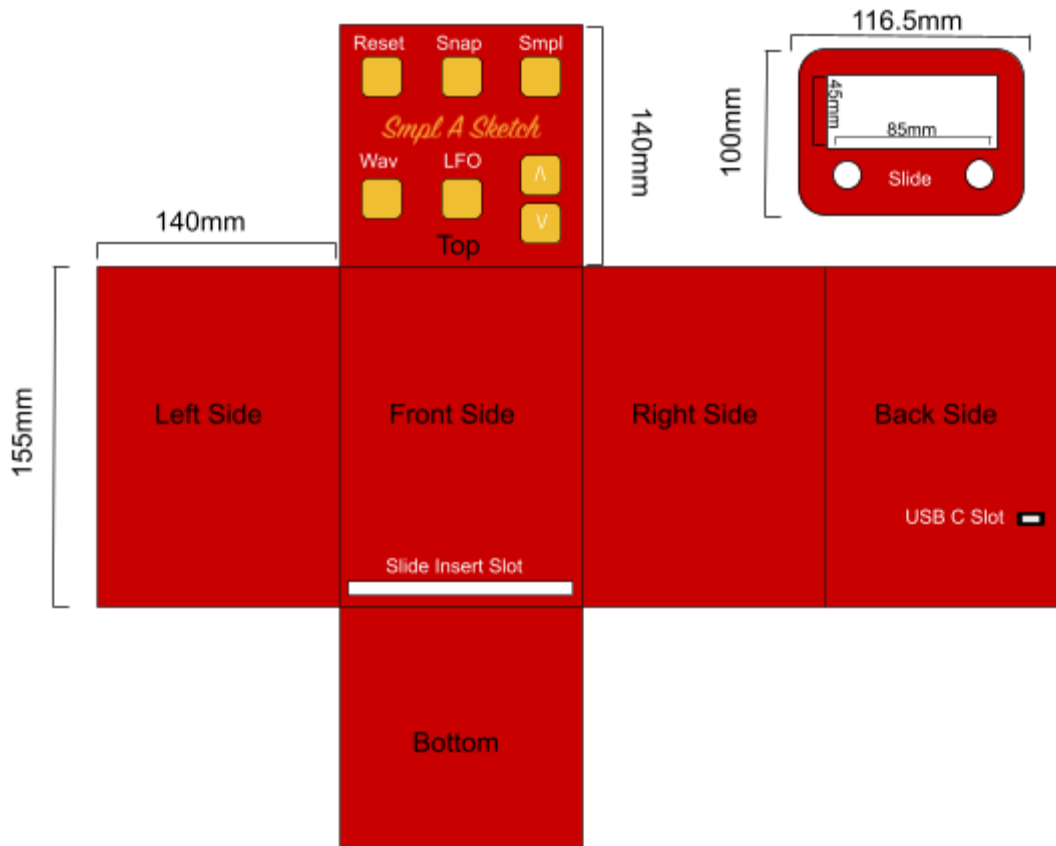
The processing UI library is also abysmal for anyone who is reading this and considering making a desktop app with processing in the future. I spent the majority of time on this project fighting with the UI builder and it hasn't been updated in 10+ years and most likely more by the time you review this.

Future Plans:

To me this project really feels done and is way more than I had ever envisioned it to be, especially now that I've recorded tutorials (I taught myself enough Adobe After Effects to animate the opening in about an hour). At a certain point I started designing the app to really be almost fully functional without the hardware with the addition of the draw mode. I didn't include a video tutorial on that yet as it's really outside the scope of the project for this class and I want to polish it up a bit more.

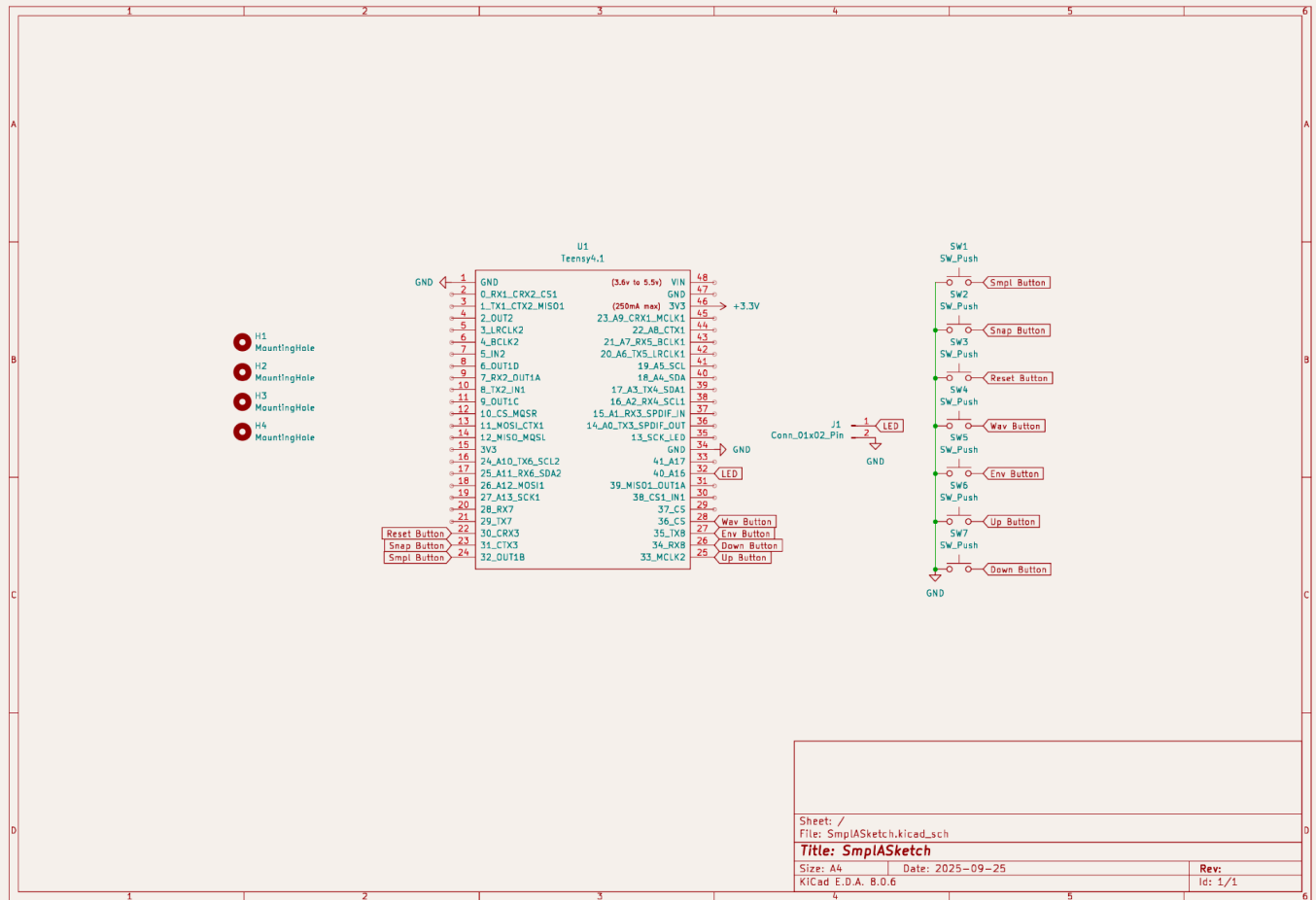
Once I iron out the issues with the draw mode on the computer I'll most likely publish the software on my website so that anyone can download it and play around with it even if they don't have the device. I know that processing has some built in web export capabilities so I may at some point also rework the app to work in a web browser. The hardware definitely is really fun to play with but I think I've somewhat unintentionally ended up with something that can also work as a fun standalone piece of software.

Completed Diagram

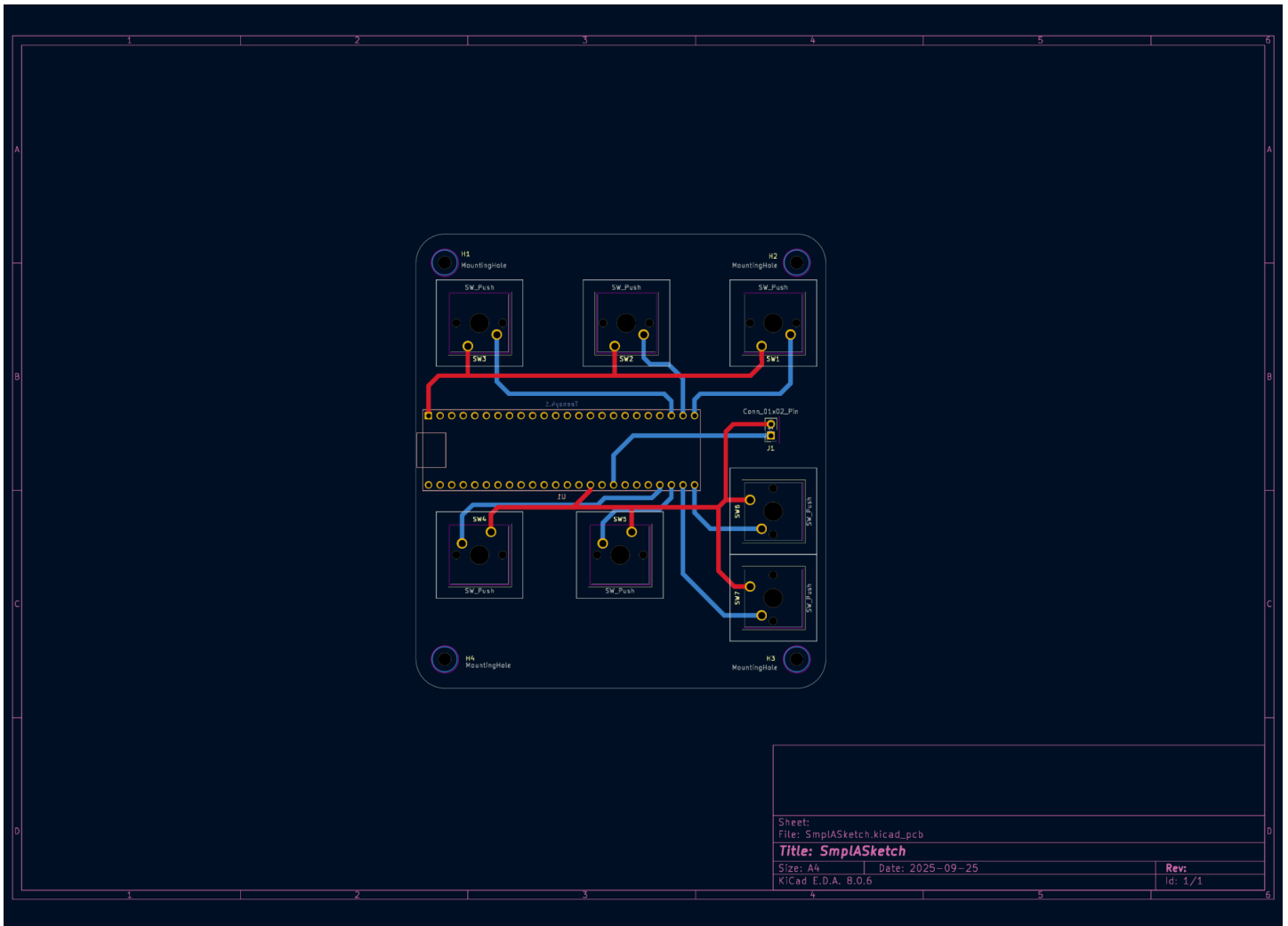


There are **9 user interface elements** on the device including 7 buttons, 1 usb port, and a slot for the user to insert the plexiglass slides. The **plexiglass slides** are drawn on with any whiteboard marker and inserted into the scan slot. For reference of what the internal device looks like scroll down to the section with the 3D prints. The **Wav and LFO buttons** must be hit first for any other button to work except for the Smpl button. The **Smpl button** plays back whatever the current sample stored is regardless of the mode selected. If the Wav button is selected then Wav mode is active, same for the LFO button and LFO mode. The **reset button** will clear the stored slide capture and replace it with a sine wave in Wav mode and reset to a straight line in LFO mode. The **snap button** will just store the drawing on the slide either in the waveform slot or LFO slot depending on the selected mode. The **up and down arrow keys** increase/decrease pitch in the wav mode and increase/decrease the speed at which the LFO plays back. Through the use of a USB hub inside the device there is one **USB C cable** to plug the device into the computer and interface with the desktop app.

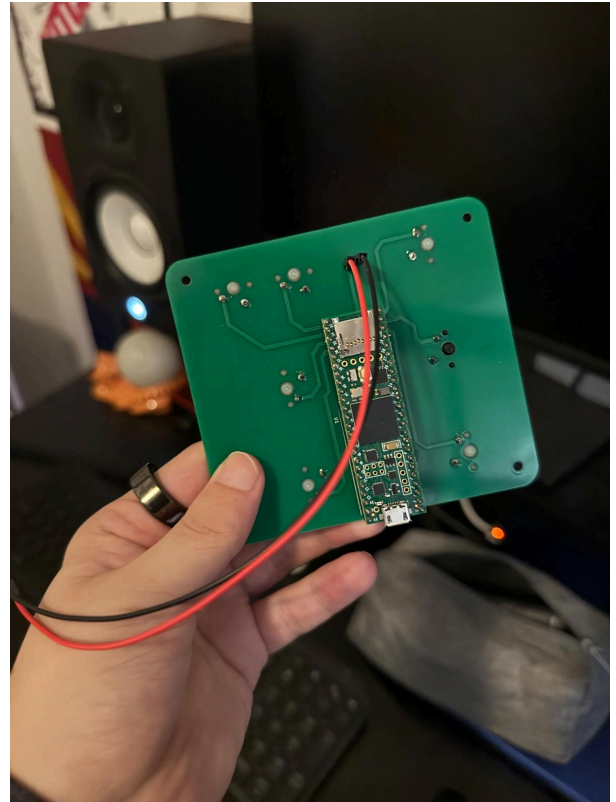
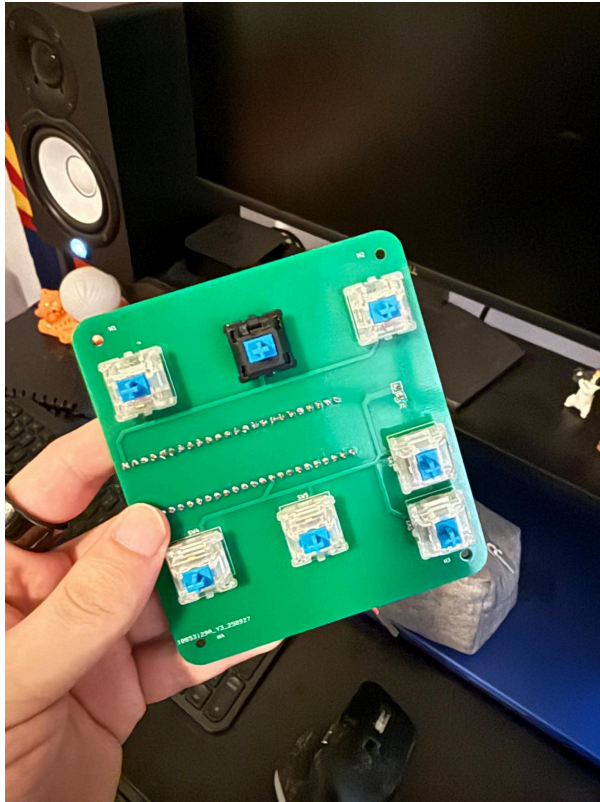
Schematic



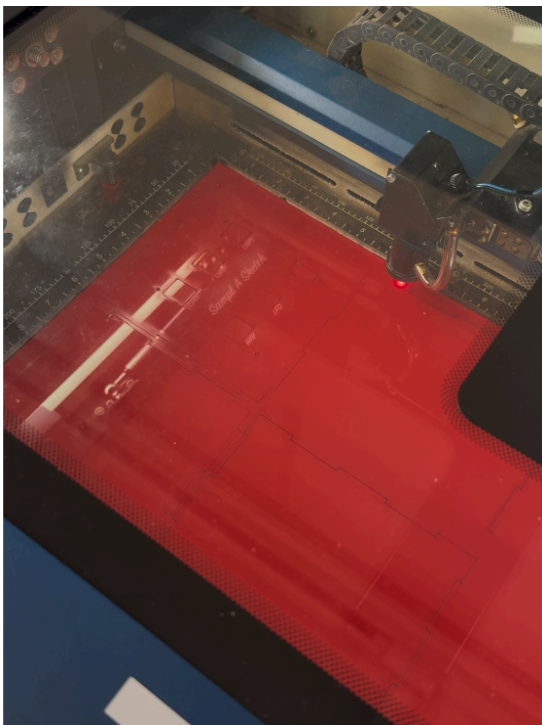
PCB Diagram



PCB Assembly



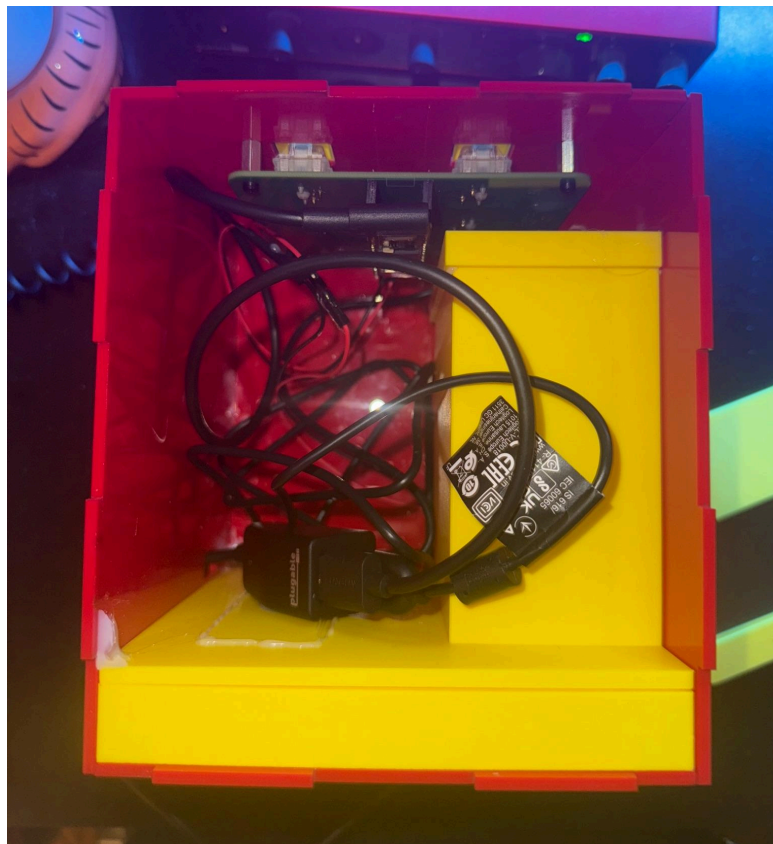
Laser Cut Housing Assembly



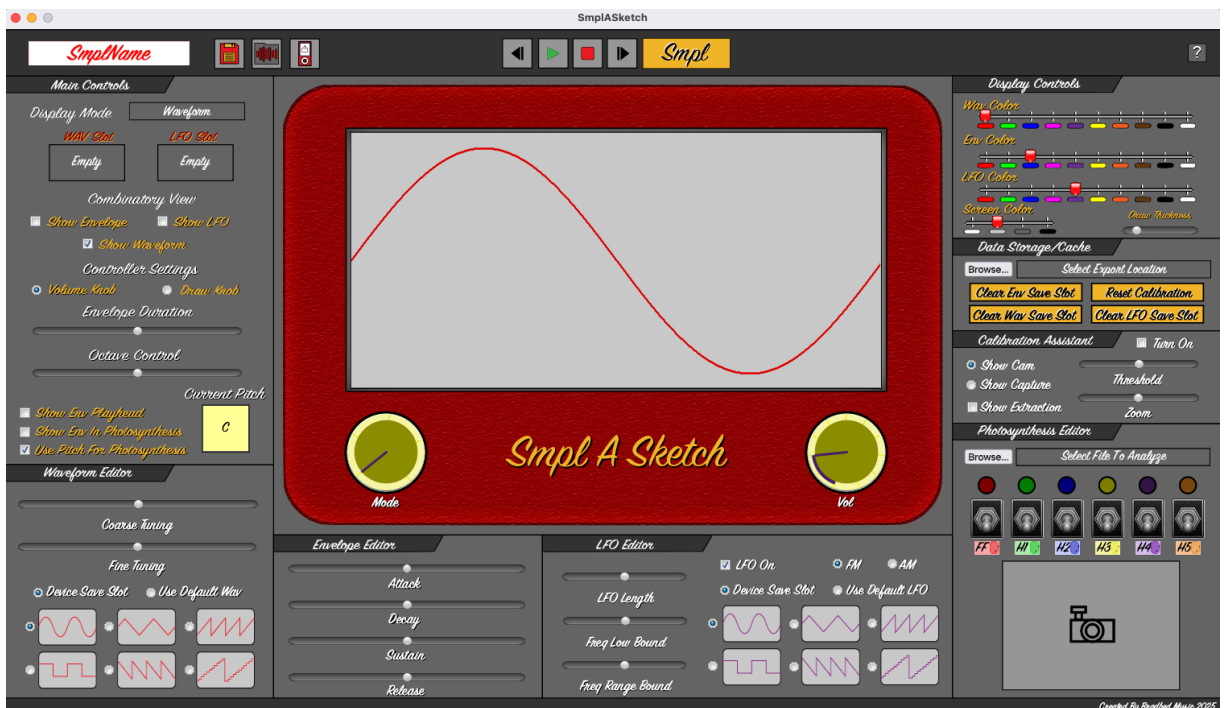
Mid-Assembly



Final Assembly Exposed



Final Product



Appendix

Raspberry Pi Headless Setup Guide: [here](#)

GitHub Page: [here](#)

Device Images Before The Final Fabrication:

